SHRI MAHAVEER COLLEGE

INTERNSHIP PROJECT REPORT

on

LOAN APPROVAL PREDICTION USING MACHINE LEARNING

Submitted in partial fulfilment of the requirements of the
Artificial Intelligence & Machine Learning Internship

Submitted by

DEEPESH TAHLANI

Under the guidance of

Mr. Md. Ameen

Naviotech Solutions Pvt Ltd

Internship Organisation

NAVIOTECH SOLUTIONS PVT LTD

Academic Year 2026 - 2027

CERTIFICATE

This is to certify that **Mr. Deepesh Tahlani**, a student of Shree Mahaveer College, has successfully completed the internship in **Artificial Intelligence & Machine Learning** at **Naviotech Solutions Pvt Ltd** during the period from 13 SEPT 2026 to 10 Oct 2026.

As part of the internship, the candidate carried out the project titled "Loan Approval Prediction using Machine Learning". The work reported in this document was done sincerely under our guidance. It is original, and the candidate showed a good understanding of data pre-processing, feature engineering, model building, evaluation and documentation.

We wish him success in all his future endeavours.

_____

Mr. Md. Ameen
Internship Guide
Naviotech Solutions Pvt Ltd

_____

Authorised Signatory
Naviotech Solutions Pvt Ltd

Date: 30/10/2026

Place: Jaipur

(Company seal)

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to everyone who supported me during my internship and in completing this project.

First, I thank my internship guide, Mr. Md. Ameen, for his valuable guidance, patient explanations and constant encouragement. His advice on how to structure a machine learning project, from understanding the data to explaining the results, helped me greatly in completing this work.

I am grateful to Naviotech Solutions Pvt Ltd for giving me the opportunity to work on practical Artificial Intelligence and Machine Learning projects. The hands-on exposure gave me real confidence in using Python and its data science libraries to solve business problems such as loan approval.

I also thank Bharati Vidyapeeth (Deemed to be University) and its faculty for providing the academic foundation on which this work is built. I thank the people who compiled and shared the Loan Prediction dataset, and the open-source community behind Python, scikit-learn, pandas, matplotlib and Jupyter, whose tools made this project possible.

Finally, I thank my family and friends for their support and motivation throughout the internship.

Deepesh Tahlani

ABSTRACT

Banks and housing finance companies receive a large number of loan applications and must decide quickly which ones to approve. A wrong approval leads to unpaid loans and financial loss, while a wrong rejection means losing a genuine customer. Manual checking is slow and can be inconsistent. This project builds a machine learning system that predicts whether a loan application will be approved or rejected from the applicant's details.

The project uses the Loan Prediction dataset of 614 past applicants, of which 422 (68.7%) were approved and 192 (31.3%) were rejected. Each record contains gender, marital status, dependents, education, employment type, applicant and co-applicant income, loan amount, loan term, credit history and property area. About 150 values are missing across seven columns. They are filled inside a scikit-learn pipeline using the median for numbers and the most frequent value for categories. New features are created: total income, log-transformed income and loan amount, the monthly instalment (EMI) and the income left after paying the EMI.

Six classifiers (Logistic Regression, KNN, SVM, Decision Tree, Random Forest and Gradient Boosting) were trained and compared using five-fold cross-validation, and Logistic Regression and Random Forest were also tuned with GridSearchCV. The best model, a tuned Logistic Regression, reached an accuracy of 86.18%, a precision of 84.00%, a recall of 98.82% and an F1-score of 90.81% on the unseen test set, compared with a baseline of 69.11% for always predicting approval. Five-fold cross-validation on the full data gave a more cautious mean accuracy of 80.30%.

The analysis shows that credit history is by far the most important factor: applicants with a good credit history were approved in about 80% of cases, against about 8% for those without. The model almost never rejects a deserving applicant, but it approves some risky ones, so a lender would raise the decision threshold. The project demonstrates a complete, explainable and reproducible machine learning workflow for a financial decision problem.

Keywords: Loan prediction, Credit risk, Classification, Logistic Regression, Random Forest, Feature engineering, Machine Learning, Python, scikit-learn.

TABLE OF CONTENTS

Certificate 2
Acknowledgement 3
Abstract 4
Chapter 1: Introduction 6
Chapter 2: Literature Review 8
Chapter 3: Methodology 10
Chapter 4: System Design 15
Chapter 5: Implementation 17
Chapter 6: Results and Discussion 20
Chapter 7: Conclusion 29
Chapter 8: Future Scope 30
References 31
Appendix: Complete Source Code 33

CHAPTER 1: INTRODUCTION

1.1 Background

Lending is the core business of every bank and housing finance company. A home loan, a personal loan or a vehicle loan is approved only after the lender checks whether the applicant is likely to repay it. Traditionally, loan officers study the application form, income proofs and credit reports and then take a decision. This takes days, depends on the experience of the officer and may differ from one officer to another.

The cost of a wrong decision is high on both sides. If a loan is approved for an applicant who later fails to repay, the lender suffers a financial loss and the loan becomes a non-performing asset. If a loan is rejected for an applicant who would have repaid it, the lender loses interest income and a good customer, and the applicant is denied a legitimate opportunity. As the number of applications grows, and as customers expect instant decisions on mobile and online platforms, lenders increasingly use data and machine learning to support these decisions.

Machine learning can learn from thousands of past applications, where the final outcome is known, and find the patterns that separate approved from rejected applications. Such a model can give a fast, consistent first assessment, which a human officer can then review. This project applies that idea to the well-known Loan Prediction dataset.

1.2 Problem Statement

Given the details of a loan applicant (personal information, income, loan amount, loan term, credit history and property area), predict whether the loan will be approved (Y) or rejected (N).

This is a binary classification problem on structured, tabular data. The dataset is small (614 records), contains missing values, has skewed numerical columns such as income, and is moderately imbalanced (about 69% approved). A good solution must handle all of these, must not give an over-optimistic picture of its performance, and should be explainable, because lending decisions must be justified to customers and regulators.

1.3 Objectives

- To understand the loan dataset and find which applicant characteristics are linked to approval.

- To handle missing values and skewed data without leaking information from the test set.
- To create new features that describe the affordability of a loan (total income, EMI and income left after EMI).
- To train and compare six classification models and tune the most promising ones.
- To evaluate the models fairly with accuracy, precision, recall, F1-score, ROC-AUC, the confusion matrix and cross-validation.
- To identify the most important features and explain the decisions of the model.
- To analyse the mistakes of the model and the effect of changing the decision threshold.
- To save the final model and use it to predict the outcome for new applicants.

1.4 Scope of the Project

The project covers the complete machine learning workflow: data loading, exploratory analysis, missing value treatment, feature engineering, model training and tuning, evaluation, interpretation and prediction. It uses a public dataset of 614 applicants. The model is intended as a decision-support tool and as a learning project; it is not a substitute for a bank's full credit appraisal, which uses many more data sources and legal checks. A web interface and integration with a banking system are suggested as future work.

1.5 Motivation

Loan approval is a classic business application of machine learning and is widely used to teach the full data science workflow, because the data is realistic and messy: it has missing values, mixed numeric and categorical columns, skewed distributions and a business meaning for every type of error. After working on text data, this project gave me the chance to work with structured financial data and to think about the cost of different mistakes, which is an important skill for a data scientist.

1.6 Organisation of the Report

Chapter 2 reviews earlier work on credit scoring. Chapter 3 explains the dataset, pre-processing, feature engineering, algorithms and evaluation measures. Chapter 4 presents the system design. Chapter 5 describes the implementation with code. Chapter 6 presents and discusses the results. Chapters 7 and 8 give the conclusion and future scope. References and the complete source code follow.

CHAPTER 2: LITERATURE REVIEW

2.1 Overview

Predicting whether a borrower will repay a loan is called credit scoring, and it is one of the oldest uses of statistics in business. This chapter summarises how credit decisions have moved from human judgement to statistical scorecards and then to machine learning.

2.2 Judgemental and Scorecard Methods

Early lending decisions were made by loan officers using rules of thumb, often summarised as the 'five Cs of credit': character, capacity, capital, collateral and conditions. These ideas are still reflected in the data used today. Character is represented by credit history, capacity by income and loan amount, capital by savings, collateral by the property, and conditions by loan terms and the property location. From the 1950s, lenders began to use statistical scorecards built with discriminant analysis and later with logistic regression, which give each applicant a score based on a weighted sum of their characteristics. Hand and Henley (1997) reviewed these statistical classification methods for consumer credit scoring and showed that logistic regression and related linear methods remain strong and practical.

2.3 Machine Learning for Credit Scoring

As computing power and data grew, researchers tested more flexible methods. West (2000) compared neural network models for credit scoring. Baesens et al. (2003) benchmarked many classification algorithms on several credit datasets and found that support vector machines and neural networks performed well, but that logistic regression was also very competitive. Khandani, Kim and Lo (2010) applied machine learning to consumer credit risk using a large bank's data and showed that tree-based methods could improve the prediction of delinquency. Lessmann et al. (2015) updated the benchmark with more datasets and methods and reported that ensemble methods such as random forests and gradient boosting generally outperform single classifiers, though the gain on small datasets can be modest.

Random forests (Breiman, 2001) and gradient boosting (Friedman, 2001) combine many decision trees to reduce error and are now standard choices for tabular data. Support vector machines (Cortes and Vapnik, 1995) find a maximum-margin boundary. K-nearest neighbours (Cover and Hart, 1967) classify an applicant by looking at the most similar past applicants. Decision trees (Quinlan, 1986) produce simple if-then rules that are easy to explain.

2.4 Imbalance, Evaluation and Explainability

Loan datasets are often imbalanced because most applications are approved or most borrowers repay. Chawla et al. (2002) proposed SMOTE, which creates synthetic examples of the minority class, as one way of dealing with this. Other methods are class weights and adjusting the decision threshold. Because the two kinds of error have different costs, researchers stress that accuracy alone is not enough and recommend precision, recall, F1-score and ROC-AUC. Explainability is equally important in lending. Lundberg and Lee (2017) introduced SHAP, which explains each prediction by showing how much every feature contributed, and many lenders prefer inherently interpretable models such as logistic regression for this reason.

Table 2.1 Comparison of credit scoring approaches

Approach	Idea	Strengths	Limitations
Judgemental (5 Cs)	Officer reviews the file	Uses human experience	Slow, subjective, inconsistent
Logistic Regression	Weighted sum of features to a probability	Fast, interpretable, stable on small data	Linear; needs good features
Decision Tree	If-then rules	Easy to explain	Unstable, overfits easily
KNN	Vote of similar applicants	Simple, no training	Sensitive to scaling and noise
SVM	Maximum-margin boundary	Strong in high dimensions	Harder to interpret
Random Forest / Gradient Boosting	Many trees combined	Capture non-linear patterns	Less transparent, need tuning
Neural networks	Layers of learned features	Very flexible	Need large data, hard to explain

2.5 Research Gap and Contribution of This Work

Many beginner implementations of this dataset train one or two models, drop all rows with missing values, and report a single accuracy from one train-test split. With only 614 records, that single number can change by several percentage points depending on the split. This project therefore keeps all records by imputing missing values inside a pipeline, compares six models under identical conditions, uses five-fold cross-validation to choose the model, tunes the best candidates, and reports precision, recall, F1-score and ROC-AUC. It also studies the errors, the feature importance and the effect of the decision threshold, so that the results are explained, not only reported.

CHAPTER 3: METHODOLOGY

3.1 Overview of the Approach

The work follows the standard machine learning workflow: (1) load and understand the data, (2) explore it with charts, (3) engineer useful features, (4) build a preprocessing pipeline that handles missing values, scaling and encoding, (5) split the data into training and test sets, (6) train and cross-validate several models, (7) tune the best candidates, (8) evaluate on unseen data, (9) interpret the model and analyse errors, and (10) save the final model and use it for new predictions.

3.2 Dataset Description

The Loan Prediction dataset comes from a practice problem of a housing finance company that gives home loans in urban, semi-urban and rural areas. The customer applies for a loan and the company decides on eligibility. The dataset has 614 records and 13 columns: 12 input columns and the target Loan_Status. There are no duplicate records. Table 3.1 describes the columns.

Table 3.1 Data dictionary

Column	Description	Type
Loan_ID	Unique ID of the application	Identifier (not used)
Gender	Male / Female	Categorical
Married	Applicant married (Yes / No)	Categorical
Dependents	Number of dependents (0, 1, 2, 3+)	Ordinal
Education	Graduate / Not Graduate	Categorical
Self_Employed	Self-employed (Yes / No)	Categorical
ApplicantIncome	Monthly income of the applicant	Numeric
CoapplicantIncome	Monthly income of the co-applicant	Numeric
LoanAmount	Loan amount in thousands	Numeric
Loan_Amount_Term	Term of the loan in months	Numeric
Credit_History	Credit history meets guidelines (1 / 0)	Categorical (binary)
Property_Area	Urban / Semiurban / Rural	Categorical
Loan_Status	Loan approved (Y) or rejected (N)	Target

Table 3.2 Summary of the dataset

Property	Value
Number of applicants	614
Approved (Y)	422 (68.7%)

Property	Value
Rejected (N)	192 (31.3%)
Mean / median applicant income	5,403 / 3,812
Mean / median loan amount (thousands)	146 / 128
Most common loan term	360 months (30 years)
Applicants with good credit history	475 (of 564 with a recorded value)
Missing entries	Gender 13, Married 3, Dependents 15, Self_Employed 32, LoanAmount 22, Loan_Amount_Term 14, Credit_History 50

The data is moderately imbalanced: about 69% of the applications were approved. This means that a model which always predicts 'approved' would already be about 69% accurate, so this figure is used as the baseline that any real model must beat.

3.3 Handling Missing Values

Seven columns contain missing values (Gender: 13, Married: 3, Dependents: 15, Self_Employed: 32, LoanAmount: 22, Loan_Amount_Term: 14, Credit_History: 50). Dropping every row with a missing value would remove a large part of an already small dataset and might bias it, so the values are filled in (imputed). Numerical columns are filled with the median, which is not affected by extreme values, and categorical columns are filled with the most frequent value (mode). Credit_History has the most missing values (50) and is the most important feature, so this choice matters; it is discussed in the limitations.

Imputation is performed inside the scikit-learn pipeline and not on the full dataset in advance. The imputer learns the median and mode only from the training data and applies them to the test data. If the full dataset were imputed first, information from the test set would leak into training and the results would look better than they really are.

3.4 Feature Engineering

Raw columns do not describe directly whether a loan is affordable, so five new features were created from existing columns:

- **TotalIncome =** ApplicantIncome + CoapplicantIncome. A couple's combined income is what repays the loan.
- **TotalIncome_log =** log(1 + TotalIncome). Income is strongly right-skewed (a few very high incomes), and the logarithm makes the distribution more symmetric.
- **LoanAmount_log =** log(1 + LoanAmount), for the same reason.

- **EMI =** LoanAmount / Loan_Amount_Term. This is the approximate monthly instalment, in thousands.
- **BalanceIncome =** TotalIncome - EMI x 1000. This shows how much income remains after paying the instalment.

The value '3+' in Dependents was converted to the number 3 so that the column can be used as a number.

3.5 Scaling and Encoding

Numerical columns are standardised (mean 0, standard deviation 1) with StandardScaler. This is necessary for distance-based and margin-based models such as KNN, SVM and Logistic Regression, because otherwise columns with large values, like income, would dominate columns with small values. Categorical columns are converted with one-hot encoding, which creates one 0/1 column for each category, so that no artificial order is imposed on categories such as Property_Area. Tree-based models do not need scaling, but it does not harm them, and using one pipeline for all models keeps the comparison fair.

3.6 Train-Test Split and Cross-Validation

The data was split into a training set (80%, 491 applicants) and a test set (20%, 123 applicants) using stratified sampling, which keeps the same share of approved and rejected loans in both parts, with a fixed random seed of 42 for reproducibility. The test set is used only for the final evaluation.

Because a test set of only 123 applicants gives a noisy estimate (one extra correct prediction changes the accuracy by 0.8 percentage points), models are also compared with five-fold stratified cross-validation on the training set. The training data is divided into five parts; each part is used once for validation while the other four are used for training, and the five scores are averaged. The best model is chosen by this cross-validated accuracy, not by the single test score, which avoids choosing a model simply because it was lucky on the test set.

3.7 Machine Learning Algorithms Used

3.7.1 Logistic Regression

Logistic Regression (Cox, 1958) computes a weighted sum of the features and passes it through the sigmoid function to produce the probability that the loan is approved. The weights are learned by minimising the log loss. Each weight shows the direction and

strength of a feature's effect, which makes the model transparent. The regularisation strength C controls how much the weights are penalised; a smaller C means a simpler model.

3.7.2 K-Nearest Neighbours (KNN)

KNN classifies an applicant by finding the k most similar applicants in the training data (here k = 11) and taking a majority vote of their outcomes. It has no real training phase, but it depends heavily on how distance is measured and works poorly when many features are noisy or irrelevant.

3.7.3 Support Vector Machine (SVM)

An SVM with an RBF kernel finds a boundary that separates the two classes with the largest possible margin, after mapping the data into a higher-dimensional space. It can model curved boundaries but is harder to interpret and sensitive to its parameters.

3.7.4 Decision Tree

A decision tree asks a sequence of yes/no questions about the features (for example 'Is the credit history 1?') and ends in a decision. The depth was limited to 4 to prevent overfitting. Trees are easy to explain but a single tree is unstable.

3.7.5 Random Forest

A Random Forest builds many decision trees on random samples of the data and random subsets of features and takes a majority vote. Averaging many different trees reduces overfitting. It also gives feature importance scores that show which features were most useful. 300 trees with a maximum depth of 5 and at least 3 samples per leaf were used initially.

3.7.6 Gradient Boosting

Gradient Boosting builds trees one after another, each new tree correcting the errors of the previous ones. 100 shallow trees (depth 2) with a learning rate of 0.05 were used. It is powerful but can overfit small datasets.

3.8 Hyper-parameter Tuning

Hyper-parameters are settings chosen before training. GridSearchCV tries every combination from a grid with five-fold cross-validation and keeps the best. It was applied to Random Forest (n_estimators in {200, 400}, max_depth in {3, 5, 7}, min_samples_leaf

in {1, 3, 5}) and to Logistic Regression (C in {0.01, 0.1, 1, 10, 100}). The search uses only the training data.

3.9 Evaluation Metrics

The positive class is 'Approved'. Let TP be approved loans predicted approved, TN rejected loans predicted rejected, FP rejected loans predicted approved (a risky loan wrongly approved) and FN approved loans predicted rejected (a good customer lost).

Table 3.3 Evaluation metrics

Metric	Formula	Meaning in this project
Accuracy	(TP + TN) / total	Share of all decisions that were correct
Precision	TP / (TP + FP)	Of the loans predicted approved, how many were really approved
Recall	TP / (TP + FN)	Of the loans that were really approved, how many were found
F1-score	2PR / (P + R)	Balance of precision and recall
ROC-AUC	Area under the ROC curve	How well the model ranks approved above rejected applicants at any threshold

For a lender the two errors are not equal. A false approval (FP) can lead to a defaulted loan, which is usually more costly than a false rejection (FN), where the lender only loses the interest on one loan. For this reason, the recall of the rejected class (how many bad applications are caught) is examined separately in Chapter 6.

CHAPTER 4: SYSTEM DESIGN

4.1 System Architecture

The system is a pipeline. Data flows through feature engineering, preprocessing, splitting, training and evaluation, and the best model is saved to a file for predicting new applications. Figure 4.1 shows the architecture.

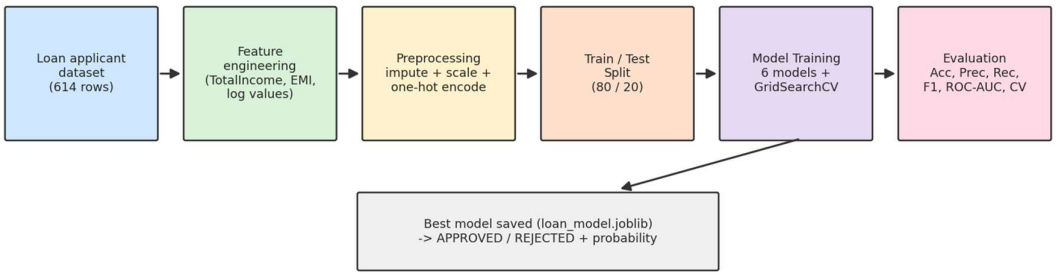


Figure 4.1 System architecture of the loan approval prediction system

4.2 Module Description

Table 4.1 Modules of the system

Module	Description	Output
Data Loading	Reads the CSV file with pandas and checks shape, duplicates and missing values	DataFrame
Exploratory Analysis	Class balance, missing values, approval rate by category, income distribution, correlation	Charts and insights
Feature Engineering	TotalIncome, log values, EMI, BalanceIncome; Dependents converted to number	Extended feature table
Preprocessing Pipeline	ColumnTransformer: impute + scale numbers, impute + one-hot encode categories	Model-ready matrix
Model Training	Six classifiers in pipelines with 5-fold cross-validation; GridSearchCV tuning	Trained models
Evaluation	Accuracy, precision, recall, F1, ROC-AUC, confusion matrix, threshold analysis	Metrics and plots
Prediction	Loads the saved pipeline, builds the same features and predicts	APPROVED / REJECTED with probability

4.3 Prediction Flow

When a new application arrives, the same feature engineering is applied (total income, EMI and so on), and the data is passed to the saved pipeline, which imputes, scales and encodes it exactly as during training and then asks the classifier for the probability of approval. A

probability of at least 0.5 is labelled APPROVED. Because the preprocessing and the model are stored together in one Pipeline object, the training and prediction steps cannot get out of sync.

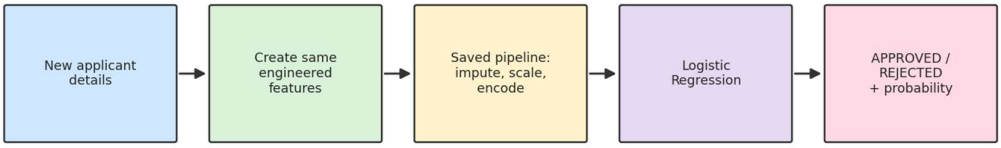


Figure 4.2 Prediction flow for a new applicant

4.4 Hardware and Software Requirements

Table 4.2 Hardware and software requirements

Item	Requirement
Processor	Any modern dual-core CPU (no GPU needed)
RAM	4 GB or more
Operating system	Windows / Linux / macOS
Language	Python 3.9 or higher
Libraries	pandas, numpy, scikit-learn, matplotlib, seaborn, joblib
Environment	Google Colab, Jupyter Notebook or VS Code
Dataset	Loan Prediction dataset (CSV, 614 records)

The whole project, including cross-validation and grid search, runs in well under a minute on an ordinary laptop.

4.5 Design Decisions

- Imputation, scaling and encoding are part of the pipeline so that they are learned from training data only (no data leakage).
- Missing values are imputed instead of dropped, to keep all 614 records.
- Models are selected by cross-validated accuracy rather than by a single small test set.
- Logistic Regression was kept in the comparison because it is the standard, explainable baseline for credit scoring.
- The decision threshold is examined separately, because the costs of the two kinds of error differ.

CHAPTER 5: IMPLEMENTATION

5.1 Development Environment

The project was implemented in Python using Google Colab, Jupyter Notebook and VS Code. pandas and numpy are used for data handling, scikit-learn for pipelines, models, tuning and metrics, matplotlib and seaborn for charts, and joblib to save the model. The code is supplied as a script (loan_prediction.py) and as a notebook (Loan_Prediction.ipynb) with outputs.

5.2 Step 1: Loading and Inspecting the Data

The CSV file is loaded with pandas. The shape, the count of each outcome, duplicates and missing values per column are checked. The script downloads the file automatically if it is not found in the project folder.

```
import numpy as np, pandas as pd

URL =
"https://raw.githubusercontent.com/emmyli/LoanPredictor/master/data/train.csv"
df = pd.read_csv(URL)
print(df.shape)                      # (614, 13)
print(df["Loan_Status"].value_counts())
print(df.isnull().sum())             # missing values per column
```

5.3 Step 2: Exploratory Data Analysis

Charts were drawn to understand the data before modelling: the loan status distribution, missing values per column, the approval rate for each category of the main categorical columns, the income distribution before and after a log transformation, and a correlation heatmap of the numerical features. These are shown in Chapter 6.

5.4 Step 3: Feature Engineering

The new columns described in Section 3.4 are added with vectorised pandas operations. The target is converted to 1 for approved and 0 for rejected. Loan_ID is dropped because it is only an identifier and carries no information.

```
df = df.drop(columns=["Loan_ID"])
df["Dependents"] = df["Dependents"].replace("3+", "3").astype(float)
df["TotalIncome"]     = df["ApplicantIncome"] + df["CoapplicantIncome"]
df["TotalIncome_log"] = np.log1p(df["TotalIncome"])
df["LoanAmount_log"]  = np.log1p(df["LoanAmount"])
df["EMI"]             = df["LoanAmount"] / df["Loan_Amount_Term"]
df["BalanceIncome"]   = df["TotalIncome"] - df["EMI"] * 1000

y = (df["Loan_Status"] == "Y").astype(int)
X = df.drop(columns=["Loan_Status"])
```

5.5 Step 4: Preprocessing Pipeline

A function builds a pipeline that first applies a ColumnTransformer (numbers: median imputation and scaling; categories: most-frequent imputation and one-hot encoding) and then a classifier. The same function is used for all models, so every model receives identical preprocessing.

```
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder

def make_pipe(clf):
    pre = ColumnTransformer([
        ("num", Pipeline([("imp", SimpleImputer(strategy="median")),
                          ("sc", StandardScaler())]), num_cols),
        ("cat", Pipeline([("imp", SimpleImputer(strategy="most_frequent")),
                          ("oh", OneHotEncoder(handle_unknown="ignore"))]),
cat_cols)])
    return Pipeline([("pre", pre), ("clf", clf)])
```

5.6 Step 5: Training with Cross-Validation

The data is split 80/20 with stratification. For each of the six models, five-fold cross-validated accuracy is computed on the training data, then the model is fitted on the full training set and evaluated on the untouched test set.

```
from sklearn.model_selection import train_test_split, StratifiedKFold,
cross_val_score

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42)
skf = StratifiedKFold(5, shuffle=True, random_state=42)

for name, clf in models.items():
    pipe = make_pipe(clf)
    cv = cross_val_score(pipe, X_train, y_train, cv=skf,
scoring="accuracy").mean()
    pipe.fit(X_train, y_train)
    y_pred = pipe.predict(X_test)
```

5.7 Step 6: Tuning and Prediction

GridSearchCV tunes Logistic Regression and Random Forest. The model with the highest cross-validated accuracy is saved with joblib. For a new applicant, the same engineered features are created and the saved pipeline returns the probability of approval.

```
from sklearn.model_selection import GridSearchCV
from sklearn.linear_model import LogisticRegression

gs = GridSearchCV(make_pipe(LogisticRegression(max_iter=1000)),
                  {"clf__C": [0.01, 0.1, 1, 10, 100]},
                  cv=skf, scoring="accuracy", n_jobs=-1).fit(X_train, y_train)
best = gs.best_estimator_             # best C = 0.1
prob = best.predict_proba(new_applicant)[0, 1]
print("APPROVED" if prob >= 0.5 else "REJECTED", prob)
```

5.8 Challenges Faced and Solutions

Table 5.1 Challenges and solutions

Challenge	Solution
Missing values in seven columns, including the key column Credit_History	Imputation inside the pipeline (median / most frequent) so that no rows are lost and no test information leaks
Skewed income and loan amount	Log transformation and standardisation
Very small dataset: one test sample changes accuracy by 0.8 points	Five-fold cross-validation for model selection; test set used only for final evaluation
Moderate class imbalance (69% approved)	Stratified split, baseline comparison, and reporting precision, recall and F1 for both classes
Choosing the best model fairly	Same pipeline, same folds and same metrics for all models; tuning only on training data
Model approves some risky applications	Threshold analysis to show the trade-off between the two kinds of error

CHAPTER 6: RESULTS AND DISCUSSION

6.1 Exploratory Data Analysis

Figure 6.1 shows that 422 of the 614 applications (68.7%) were approved and 192 (31.3%) were rejected.

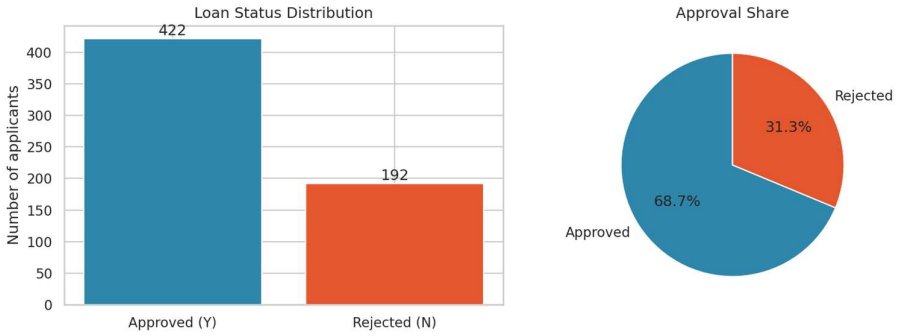


Figure 6.1 Distribution of loan status

Figure 6.2 shows the number of missing values. Credit_History (50) and Self_Employed (32) have the most. The fact that Credit_History, the most important feature, is missing for 8% of applicants shows why careful imputation matters.

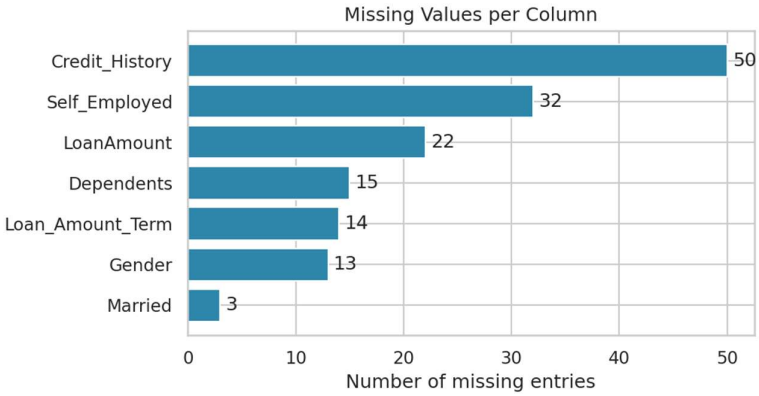


Figure 6.2 Missing values per column

Figure 6.3 compares the approval rate across the main categorical features. The strongest pattern by far is credit history: 79.6% of applicants with a good credit history were approved, compared with only 7.9% of those without. Property area also matters (Semiurban 76.8%, Urban 65.8%, Rural 61.5%). Graduates (70.8%) and married applicants (71.6%) were approved somewhat more often than non-graduates (61.2%) and unmarried applicants (62.9%). The differences by gender (67% for women, 69.3% for men) and by self-employment (68.3% against 68.6%) are very small.

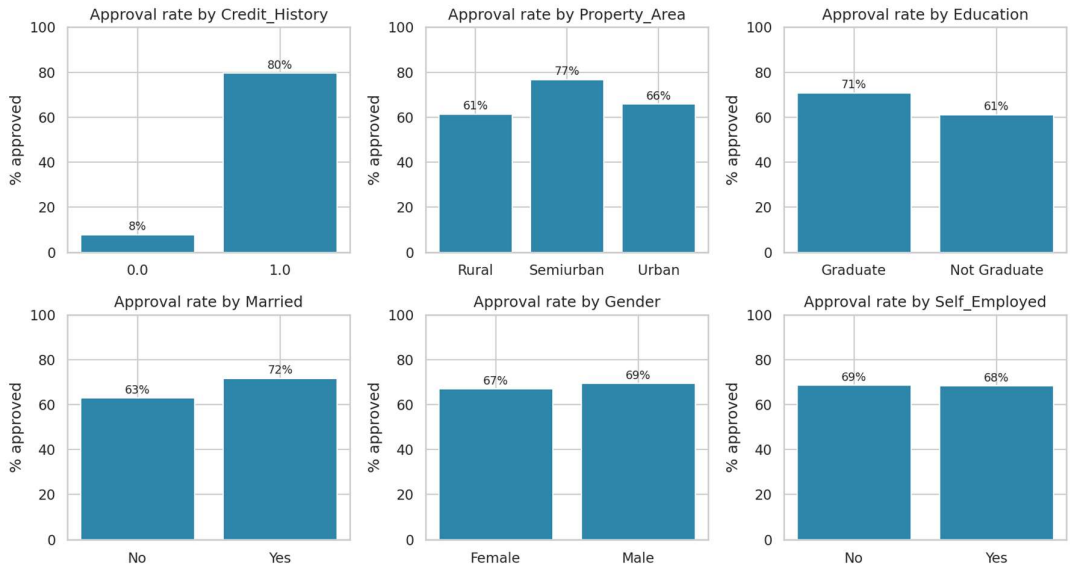


Figure 6.3 Approval rate by category

Figure 6.4 shows that applicant income is strongly right-skewed: most applicants earn a moderate income while a few earn very high amounts (the maximum is 81,000). After the log transformation the distribution is much closer to a normal shape, which is better for linear models.

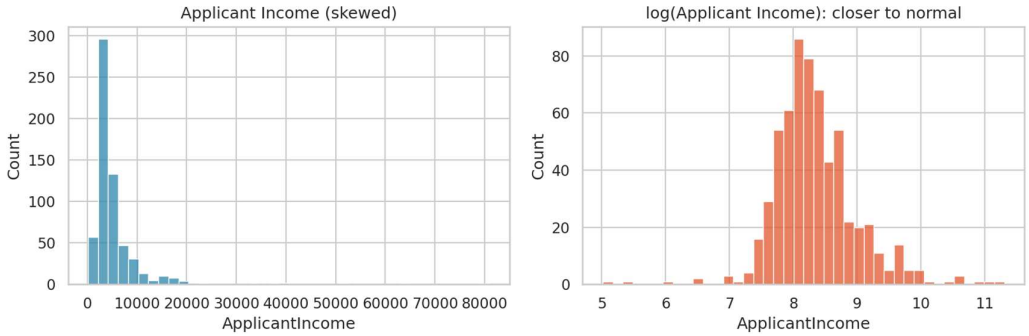


Figure 6.4 Applicant income before and after log transformation

Figure 6.5 shows the correlations between the numerical features and the outcome. Income-related features are correlated with each other, which is expected, but none of them has a meaningful linear correlation with loan approval (all are below 0.06 in absolute value), which already suggests that income alone does not decide the outcome.

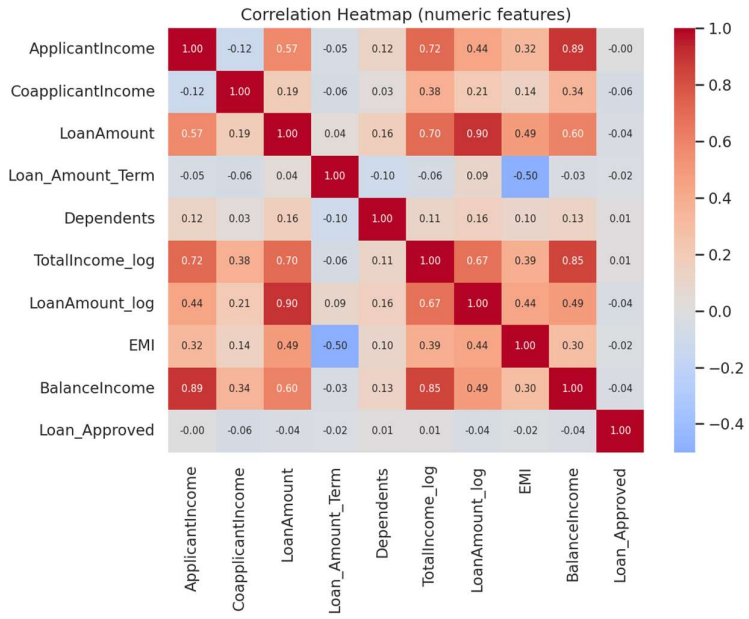


Figure 6.5 Correlation heatmap of numerical features

6.2 Model Comparison

All models were trained on the same 491 training applicants and tested on the same 123 unseen applicants. Table 6.1 shows the results sorted by cross-validated accuracy, which was used to choose the best model. The baseline of always predicting 'approved' gives 69.11%.

Table 6.1 Performance of all models

Model	CV Acc.	Test Acc.	Precision	Recall	F1	ROC-AUC
Logistic Regression (tuned)	79.83%	86.18%	84.00%	98.82%	90.81%	0.854
Random Forest (tuned)	79.83%	85.37%	83.17%	98.82%	90.32%	0.849
Logistic Regression	79.62%	85.37%	83.84%	97.65%	90.22%	0.858
Random Forest	79.42%	83.74%	82.83%	96.47%	89.13%	0.833
SVM (RBF)	79.02%	84.55%	82.35%	98.82%	89.84%	0.824
Gradient Boosting	79.01%	82.93%	82.65%	95.29%	88.52%	0.812
Decision Tree	77.39%	84.55%	83.67%	96.47%	89.62%	0.768
KNN	73.11%	82.11%	80.00%	98.82%	88.42%	0.763

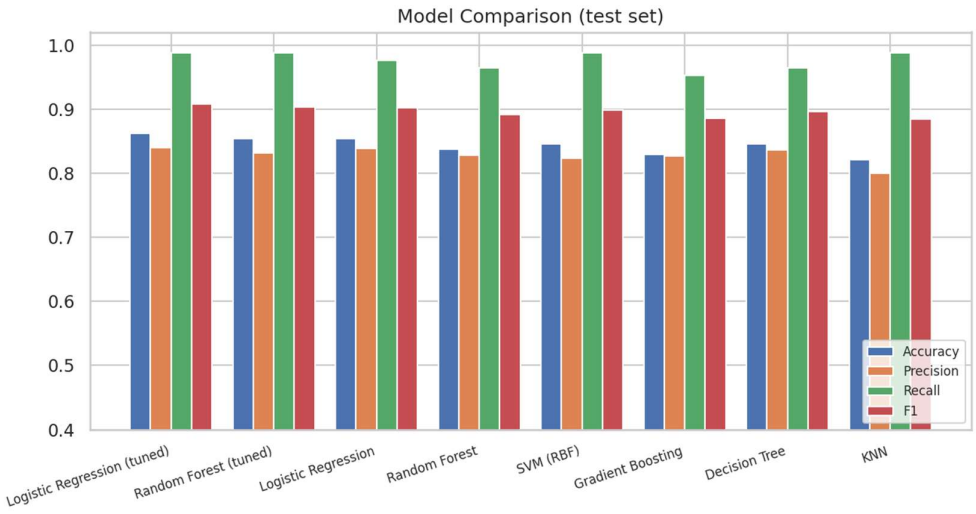


Figure 6.6 Comparison of the models on the test set

Every model beats the 69.11% baseline clearly, with test accuracies between 82.11% and 86.18%. The differences between the top models are small: the cross-validated accuracies of the top six models lie within about one percentage point (79.01% to 79.83%), which is within the noise of a dataset of this size. The tuned Logistic Regression and the tuned Random Forest tie on cross-validated accuracy (79.83%); Logistic Regression has the higher test F1-score and the higher ROC-AUC, and it is far easier to explain, so it was selected as the final model. KNN is clearly the weakest, which is expected because many features are noisy or irrelevant, and the single Decision Tree and KNN have the lowest ROC-AUC values, which shows that they rank applicants less reliably.

An important lesson is that a more complex model (Gradient Boosting, Random Forest) did not beat the simple Logistic Regression. When a dataset is small and one feature (credit history) dominates, simple models are often just as good.

6.3 Detailed Results of the Best Model (Logistic Regression (tuned))

The best parameters found by GridSearchCV were C = 0.1, a fairly strong regularisation that keeps the model simple.

6.3.1 Confusion Matrix

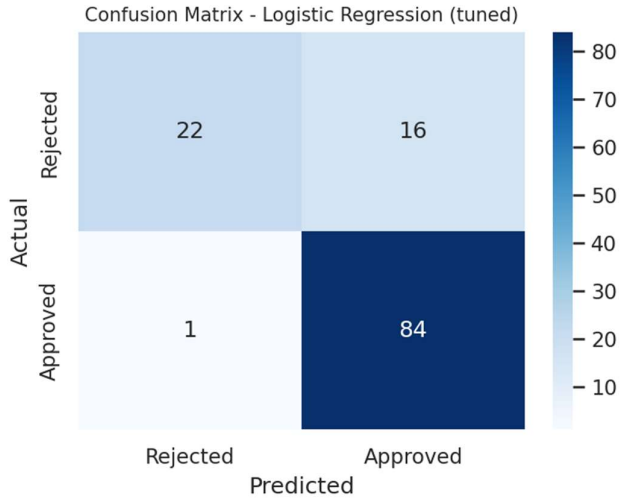


Figure 6.7 Confusion matrix of the best model

Table 6.2 Confusion matrix values

	Predicted Rejected	**Predicted Approved**
Actual Rejected	22 (true negatives)	16 (false positives)
Actual Approved	1 (false negatives)	84 (true positives)

Of the 85 applications that were really approved, the model approved 84 and wrongly rejected only 1. Of the 38 that were really rejected, it correctly rejected 22 but wrongly approved 16. The model is therefore very good at recognising good applicants but it is lenient: it lets through many applications that the lender actually rejected.

6.3.2 Classification Report

```
precision    recall  f1-score   support

    Rejected     0.9565    0.5789    0.7213        38
    Approved     0.8400    0.9882    0.9081        85

    accuracy                         0.8618       123
   macro avg     0.8983    0.7836    0.8147       123
weighted avg     0.8760    0.8618    0.8504       123
```

The recall of the Approved class is 98.82%, but the recall of the Rejected class is only 57.89%. The precision of the Rejected class is high (95.65%): when the model does reject an application, it is almost always right. For a lender, the weak point is the 16 rejected applications that were approved.

6.3.3 ROC Curves

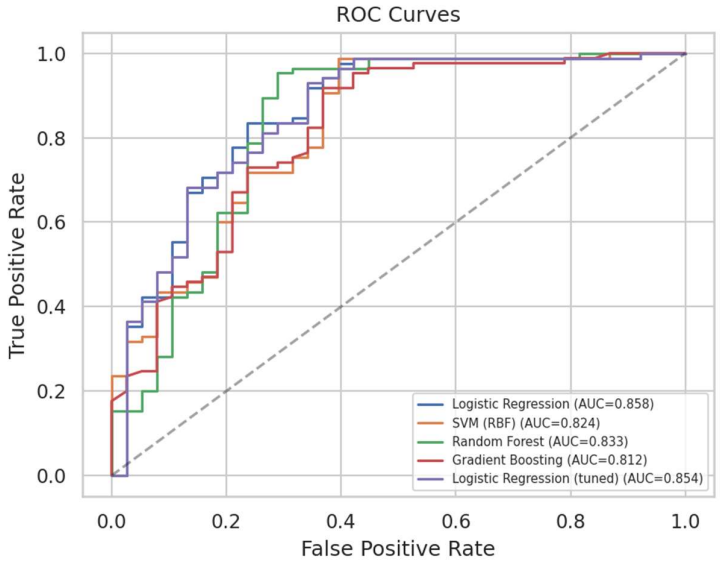


Figure 6.8 ROC curves of the main models

The ROC-AUC of the best model is 0.854, so it ranks approved applicants above rejected ones correctly about 85% of the time. A perfect model would score 1.0 and random guessing 0.5.

6.3.4 Cross-Validation on the Full Data

Five-fold stratified cross-validation on all 614 records gave accuracies of 82.11%, 81.30%, 79.67%, 75.61%, 82.79%, with a mean of 80.30%. This is lower than the 86.18% on the test set. The gap is mainly due to the small test set (123 applicants), which happened to be a little easier than average. The cross-validated figure of about 80% is the more realistic estimate of how the model will perform on new applicants, and this is the number that should be quoted with the 86% test score.

6.4 Which Features Matter Most?

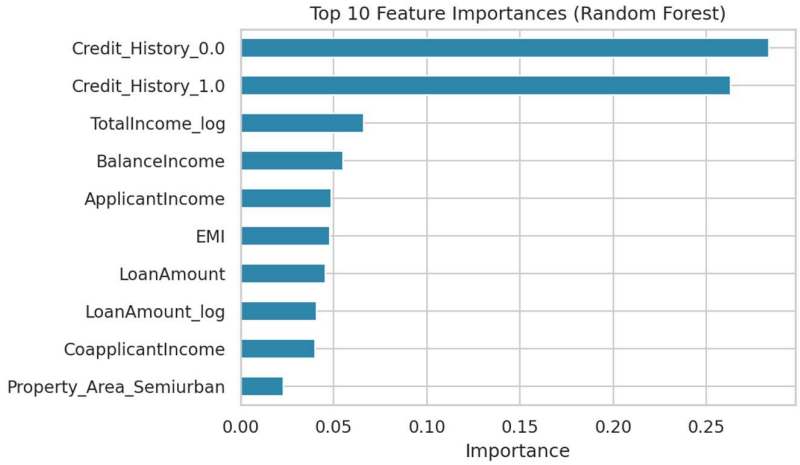


Figure 6.9 Top 10 feature importances (Random Forest)

The Random Forest importance scores confirm the exploratory analysis. The two credit-history columns (the two categories of the same feature after one-hot encoding) together account for about 55% of the total importance, far more than any other feature. Next come the income and loan features: TotalIncome_log, BalanceIncome, ApplicantIncome, EMI and LoanAmount. Two of these, TotalIncome_log and BalanceIncome, are engineered features, which shows that the feature engineering step added useful information.

The coefficients of the Logistic Regression give the direction of each effect. After scaling, the largest are Credit_History = 1 (+1.10, pushes towards approval) and Credit_History = 0 (-1.10, pushes towards rejection). Other positive effects are Semiurban property (+0.32), being married (+0.18) and being a graduate (+0.15). Negative effects are a rural property (-0.22), being unmarried (-0.18) and not being a graduate (-0.15). Credit history is about three times stronger than any other single factor.

6.5 Effect of the Decision Threshold

By default, an application is approved when the predicted probability is at least 0.5. Raising the threshold makes the model stricter. Table 6.3 shows what happens on the test set.

Table 6.3 Effect of the decision threshold

Threshold	Accuracy	Rejected precision	Rejected recall	Approved precision	Approved recall
0.50	86.18%	95.65%	57.89%	84.00%	98.82%
0.60	85.37%	91.67%	57.89%	83.84%	97.65%
0.70	78.86%	64.29%	71.05%	86.42%	82.35%

Threshold	Accuracy	Rejected precision	Rejected recall	Approved precision	Approved recall
0.75	69.11%	50.00%	86.84%	91.23%	61.18%
0.80	52.85%	39.36%	97.37%	96.55%	32.94%

At a threshold of 0.70 the model catches 71.1% of the applications that should be rejected (up from 57.9%) while still approving 82.4% of the good ones, at the cost of overall accuracy falling from 86.2% to 78.9%. At 0.80 it catches 97.4% of the bad applications but approves only 32.9% of the good ones, which would make the lender lose most of its business. The right threshold depends on the lender's own cost of a defaulted loan compared with the profit from a good loan, so this is a business decision, not a purely technical one. It also shows why accuracy alone is a poor guide: the model with the highest accuracy is not the one that is safest for the lender.

6.6 Error Analysis

The model made 16 false approvals and 1 false rejection on the test set.

False approvals (rejected loans that the model approved). All 16 of them had a good credit history (Credit_History = 1) and the model gave them approval probabilities between about 0.60 and 0.87. Among the 38 rejected applications in the test set, 21 had no good credit history and the model rejected essentially all of them; the other 17 had a good credit history, and the model approved 16 of these. In other words, the model has learnt the rule 'good credit history means approve' very well, but the available columns do not contain the reasons why the lender rejected some applicants with a good credit history. Examples include an applicant with a very low income of 150 plus a co-applicant income of 1,800 asking for 135 thousand, and an applicant with a high income of 18,333 asking for a very large loan of 500 thousand.

False rejection. The only approved loan that the model rejected belonged to an applicant with no good credit history (income 4,917, loan 130 thousand, rural property). The model gave it a probability of 0.19, which is reasonable given that only about 8% of such applicants were approved in the data.

These errors point to a limit of the data and not of the algorithm. Information such as the applicant's existing debts, the value of the property, employment history and the bank's internal rules is not included, so no model can fully explain these decisions.

6.7 Final Prediction Output

The saved model was used to predict four new hypothetical applicants. Changing only the credit history of the same applicant flips the decision from approved to rejected, which agrees with the analysis above. The low-income applicant asking for a large loan with no credit record is rejected, and the high-income applicant with a small loan and a good credit record is approved.

```
FINAL PREDICTIONS
--------------------------------------------------------------------------------
[APPROVED ] P(approve)=0.84  Graduate, married, good credit, income 6000
[REJECTED ] P(approve)=0.38  Same profile but bad credit history
[REJECTED ] P(approve)=0.12  Low income, large loan, no credit record
[APPROVED ] P(approve)=0.78  High income, small loan, good credit
```

6.8 Discussion and Limitations

The results show that a simple, well-prepared pipeline reaches about 80% to 86% accuracy on this problem, about 11 to 17 points above the baseline. Most of the predictive power comes from credit history, and the choice of algorithm matters much less than good preprocessing and careful evaluation.

The limitations of the study should be stated clearly. First, the dataset is small (614 records), so all scores have a wide margin of error, as the variation between cross-validation folds (75.6% to 82.8%) shows. Second, missing credit history values were filled with the most frequent value (good credit), which may not be right for every applicant. Third, the data comes from one lender and one period, and the model may not work for other lenders or later years. Fourth, the dataset contains gender and marital status. These attributes show only a weak association with approval in the data, but a real lending system must be tested for fairness and must follow the law on the use of such attributes. Finally, the model predicts what the lender decided in the past, which is not the same as predicting whether the borrower would really have repaid.

CHAPTER 7: CONCLUSION

This project developed a machine learning system that predicts whether a loan application will be approved. The full workflow was carried out: exploratory analysis, handling of missing values, feature engineering, building a leak-free preprocessing pipeline, training and cross-validating six models, tuning the best candidates, evaluating them on unseen data, interpreting the model and analysing its errors and its decision threshold.

The final model, a tuned Logistic Regression, achieved an accuracy of 86.18%, a precision of 84.00%, a recall of 98.82%, an F1-score of 90.81% and a ROC-AUC of 0.854 on the test set, against a baseline of 69.11%. Five-fold cross-validation on the full data gave a mean accuracy of 80.30%, which is the more cautious estimate of real performance.

The main findings are the following. First, credit history is by far the most important factor, followed by income-related features and the property area. Second, the engineered features (total income, EMI and balance income) added useful information. Third, on a small dataset a simple and explainable model performed as well as more complex ones. Fourth, the model is excellent at recognising good applicants but lenient towards risky ones, so a lender should raise the decision threshold, accepting a lower overall accuracy for a higher rejection recall. Fifth, with only 614 records, a single test score is unreliable, and cross-validation should always be used.

Through this project I learnt how to work with messy tabular data, how to avoid data leakage with pipelines, how to compare models fairly, and how to relate evaluation metrics to the business cost of different errors. These are skills I will use in future data science work.

CHAPTER 8: FUTURE SCOPE

The system can be improved and extended in several directions:

- **More data and richer features.** Collect more applications and more features, such as existing debts, savings, employment history, property value and credit score, which would address the limits found in the error analysis.
- **Imbalance and cost-sensitive learning.** Use SMOTE, class weights or cost-sensitive learning to improve the detection of risky applications, and choose the decision threshold using the lender's real cost of a default.
- **Advanced models.** Try XGBoost, LightGBM and CatBoost, which often perform well on tabular data, and stacking of several models.
- **Explainable AI.** Use SHAP or LIME to explain each decision to the applicant and the loan officer, and to give reasons for rejection.
- **Fairness audit.** Test the model for bias with respect to gender, marital status and region, and apply fairness techniques, as required for responsible lending.
- **Web application and API.** Build a Streamlit or Flask web application where a loan officer enters the details of an applicant and sees the prediction and probability, and expose the model as a REST API.
- **Monitoring and retraining.** Monitor the model after deployment and retrain it regularly, because customer behaviour and the economy change over time.
- **Wider prediction tasks.** Predict not only approval but also the risk of default, the suitable loan amount and the interest rate.

REFERENCES

[1] Hand, D. J., and Henley, W. E. (1997). Statistical classification methods in consumer credit scoring: a review. Journal of the Royal Statistical Society, Series A, 160(3), 523-541.

[2] Baesens, B., Van Gestel, T., Viaene, S., Stepanova, M., Suykens, J., and Vanthienen, J. (2003). Benchmarking state-of-the-art classification algorithms for credit scoring. Journal of the Operational Research Society, 54(6), 627-635.

[3] Lessmann, S., Baesens, B., Seow, H.-V., and Thomas, L. C. (2015). Benchmarking state-of-the-art classification algorithms for credit scoring: an update of research. European Journal of Operational Research, 247(1), 124-136.

[4] Khandani, A. E., Kim, A. J., and Lo, A. W. (2010). Consumer credit-risk models via machine-learning algorithms. Journal of Banking and Finance, 34(11), 2767-2787.

[5] West, D. (2000). Neural network credit scoring models. Computers and Operations Research, 27(11-12), 1131-1152.

[6] Breiman, L. (2001). Random forests. Machine Learning, 45(1), 5-32.

[7] Friedman, J. H. (2001). Greedy function approximation: a gradient boosting machine. Annals of Statistics, 29(5), 1189-1232.

[8] Cortes, C., and Vapnik, V. (1995). Support-vector networks. Machine Learning, 20(3), 273-297.

[9] Cox, D. R. (1958). The regression analysis of binary sequences. Journal of the Royal Statistical Society, Series B, 20(2), 215-242.

[10] Quinlan, J. R. (1986). Induction of decision trees. Machine Learning, 1(1), 81-106.

[11] Cover, T., and Hart, P. (1967). Nearest neighbor pattern classification. IEEE Transactions on Information Theory, 13(1), 21-27.

[12] Chawla, N. V., Bowyer, K. W., Hall, L. O., and Kegelmeyer, W. P. (2002). SMOTE: synthetic minority over-sampling technique. Journal of Artificial Intelligence Research, 16, 321-357.

[13] Lundberg, S. M., and Lee, S.-I. (2017). A unified approach to interpreting model predictions. Advances in Neural Information Processing Systems 30 (NIPS 2017).

[14] Hosmer, D. W., Lemeshow, S., and Sturdivant, R. X. (2013). Applied Logistic Regression (3rd ed.). Wiley.

[15] Pedregosa, F., et al. (2011). Scikit-learn: machine learning in Python. Journal of Machine Learning Research, 12, 2825-2830.

[16] McKinney, W. (2010). Data structures for statistical computing in Python. Proceedings of the 9th Python in Science Conference, 51-56.

[17] Hunter, J. D. (2007). Matplotlib: a 2D graphics environment. Computing in Science and Engineering, 9(3), 90-95.

[18] Analytics Vidhya. Loan Prediction practice problem (Dream Housing Finance dataset). The copy of the training data used in this project: https://github.com/emmyli/LoanPredictor (data/train.csv).

APPENDIX: COMPLETE SOURCE CODE

The complete script (loan_prediction.py) used in this project is listed below. The same code, in step-by-step form with outputs, is available in the notebook Loan_Prediction.ipynb.

```
"""
Loan Approval Prediction using Machine Learning
Author     : Shashank Deshmukh
Internship : Naviotech Solutions Pvt Ltd (AI & ML)

Pipeline: load data -> EDA -> feature engineering -> preprocessing pipeline
          -> train 6 models -> cross-validate -> tune -> evaluate -> save ->
predict
"""

import json
import os
import urllib.request
import warnings

import joblib
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns
from sklearn.compose import ColumnTransformer
from sklearn.ensemble import GradientBoostingClassifier, RandomForestClassifier
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (accuracy_score, classification_report,
                             confusion_matrix, f1_score, precision_score,
                             recall_score, roc_auc_score, roc_curve)
from sklearn.model_selection import (GridSearchCV, StratifiedKFold,
                                     cross_val_score, train_test_split)
from sklearn.neighbors import KNeighborsClassifier
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier

warnings.filterwarnings("ignore")
sns.set_theme(style="whitegrid")
RANDOM_STATE = 42
BLUE, ORANGE = "#2e86ab", "#e4572e"

# ---------------------------------------------------------------- 1. Load
DATA_FILE = "loan.csv"
DATA_URL =
"https://raw.githubusercontent.com/emmyli/LoanPredictor/master/data/train.csv"
if not os.path.exists(DATA_FILE):  # download automatically if the file is missing
    urllib.request.urlretrieve(DATA_URL, DATA_FILE)

df = pd.read_csv(DATA_FILE)
print("Dataset shape:", df.shape)
print(df.head(), "\n")
print("Loan status counts:\n", df["Loan_Status"].value_counts(), "\n")
missing = df.isnull().sum()
print("Missing values:\n", missing[missing > 0], "\n")
print("Duplicate rows:", df.duplicated().sum(), "\n")
print(df.describe().round(1), "\n")

# ---------------------------------------------------------------- 2. EDA plots
```

```
fig, ax = plt.subplots(1, 2, figsize=(11, 4))
counts = df["Loan_Status"].value_counts()
ax[0].bar(["Approved (Y)", "Rejected (N)"], [counts["Y"], counts["N"]],
color=[BLUE, ORANGE])
ax[0].set_title("Loan Status Distribution")
ax[0].set_ylabel("Number of applicants")
for i, v in enumerate([counts["Y"], counts["N"]]):
    ax[0].text(i, v + 6, str(v), ha="center")
ax[1].pie([counts["Y"], counts["N"]], labels=["Approved", "Rejected"],
autopct="%1.1f%%",
          colors=[BLUE, ORANGE], startangle=90)
ax[1].set_title("Approval Share")
plt.tight_layout(); plt.savefig("fig_class_distribution.png", dpi=150);
plt.close()

mv = missing[missing > 0].sort_values()
plt.figure(figsize=(7, 3.8))
plt.barh(mv.index, mv.values, color=BLUE)
for i, v in enumerate(mv.values):
    plt.text(v + 0.5, i, str(v), va="center")
plt.title("Missing Values per Column"); plt.xlabel("Number of missing entries")
plt.tight_layout(); plt.savefig("fig_missing_values.png", dpi=150); plt.close()

df["approved"] = (df["Loan_Status"] == "Y").astype(int)
cats = ["Credit_History", "Property_Area", "Education", "Married", "Gender",
"Self_Employed"]
fig, axes = plt.subplots(2, 3, figsize=(12, 6.5))
rates = {}
for a, c in zip(axes.ravel(), cats):
    r = df.groupby(c)["approved"].mean() * 100
    rates[c] = {str(k): round(v, 1) for k, v in r.items()}
    a.bar([str(k) for k in r.index], r.values, color=BLUE)
    a.set_title(f"Approval rate by {c}"); a.set_ylim(0, 100); a.set_ylabel("%
approved")
    for i, v in enumerate(r.values):
        a.text(i, v + 2, f"{v:.0f}%", ha="center", fontsize=9)
plt.tight_layout(); plt.savefig("fig_approval_rates.png", dpi=150); plt.close()

fig, ax = plt.subplots(1, 2, figsize=(11, 3.8))
sns.histplot(df["ApplicantIncome"], bins=40, color=BLUE, ax=ax[0])
ax[0].set_title("Applicant Income (skewed)")
sns.histplot(np.log1p(df["ApplicantIncome"]), bins=40, color=ORANGE, ax=ax[1])
ax[1].set_title("log(Applicant Income): closer to normal")
plt.tight_layout(); plt.savefig("fig_income_distribution.png", dpi=150);
plt.close()

# ------------------------------------------------------------------ 3. Feature
engineering
df = df.drop(columns=["Loan_ID", "approved"])
df["Dependents"] = df["Dependents"].replace("3+", "3").astype(float)
df["TotalIncome"] = df["ApplicantIncome"] + df["CoapplicantIncome"]
df["TotalIncome_log"] = np.log1p(df["TotalIncome"])
df["LoanAmount_log"] = np.log1p(df["LoanAmount"])
df["EMI"] = df["LoanAmount"] / df["Loan_Amount_Term"]          # thousands per
month
df["BalanceIncome"] = df["TotalIncome"] - df["EMI"] * 1000     # income left after
EMI
y = (df["Loan_Status"] == "Y").astype(int)
X = df.drop(columns=["Loan_Status"])

num_cols = ["ApplicantIncome", "CoapplicantIncome", "LoanAmount",
"Loan_Amount_Term", "Dependents",
            "TotalIncome_log", "LoanAmount_log", "EMI", "BalanceIncome"]
cat_cols = ["Gender", "Married", "Education", "Self_Employed", "Credit_History",
"Property_Area"]

num_corr = pd.concat([X[num_cols], y.rename("Loan_Approved")], axis=1).corr()
plt.figure(figsize=(8, 6.5))
```

```
sns.heatmap(num_corr, annot=True, fmt=".2f", cmap="coolwarm", center=0,
annot_kws={"size": 7})
plt.title("Correlation Heatmap (numeric features)")
plt.tight_layout(); plt.savefig("fig_correlation.png", dpi=150); plt.close()

# ---------------------------------------------------------------- 4. Split
X train, X test, y train, y test = train test split(
    X, y, test_size=0.2, stratify=y, random_state=RANDOM_STATE)
print("Train size:", len(X_train), "| Test size:", len(X_test), "\n")
baseline = max(y_test.mean(), 1 - y_test.mean())
print(f"Baseline (always predict 'Approved'): {baseline:.4f}\n")



def make_pipe(clf):
    pre = ColumnTransformer([
        ("num", Pipeline([("imp", SimpleImputer(strategy="median")),
                          ("sc", StandardScaler())]), num_cols),
        ("cat", Pipeline([("imp", SimpleImputer(strategy="most_frequent")),
                          ("oh", OneHotEncoder(handle_unknown="ignore"))]),
cat_cols),
    ])
    return Pipeline([("pre", pre), ("clf", clf)])



models = {
    "Logistic Regression": LogisticRegression(max_iter=1000),
    "KNN": KNeighborsClassifier(n_neighbors=11),
    "SVM (RBF)": SVC(probability=True, random_state=RANDOM_STATE),
    "Decision Tree": DecisionTreeClassifier(max_depth=4,
random_state=RANDOM_STATE),
    "Random Forest": RandomForestClassifier(n estimators=300, max depth=5,
min_samples_leaf=3,
                                            random_state=RANDOM_STATE),
    "Gradient Boosting": GradientBoostingClassifier(n_estimators=100,
learning_rate=0.05,
                                                    max_depth=2,
random_state=RANDOM_STATE),
}

skf = StratifiedKFold(5, shuffle=True, random_state=RANDOM_STATE)
rows, fitted, preds, scores = [], {}, {}, {}
for name, clf in models.items():
    pipe = make_pipe(clf)
    cv = cross_val_score(pipe, X_train, y_train, cv=skf, scoring="accuracy")
    pipe.fit(X_train, y_train)
    yp = pipe.predict(X_test)
    s = pipe.predict_proba(X_test)[:, 1]
    fitted[name], preds[name], scores[name] = pipe, yp, s
    rows.append({
        "Model": name,
        "CV_Accuracy": cv.mean(),
        "Accuracy": accuracy_score(y_test, yp),
        "Precision": precision_score(y_test, yp),
        "Recall": recall_score(y_test, yp),
        "F1": f1_score(y_test, yp),
        "ROC_AUC": roc_auc_score(y_test, s),
    })
    r = rows[-1]
    print(f"{name:20s} cv={r['CV_Accuracy']:.4f} acc={r['Accuracy']:.4f}
prec={r['Precision']:.4f} "
          f"rec={r['Recall']:.4f} f1={r['F1']:.4f} auc={r['ROC_AUC']:.4f}")

# ---------------------------------------------------------------- 5. Hyper-
parameter tuning
print("\nTuning Random Forest and Logistic Regression with GridSearchCV (5-
fold)...")
grids = {
    "Random Forest (tuned)": (RandomForestClassifier(random_state=RANDOM_STATE),
```

```
                                {"clf__n_estimators": [200, 400], "clf__max_depth":
[3, 5, 7],
                                 "clf__min_samples_leaf": [1, 3, 5]}),
    "Logistic Regression (tuned)": (LogisticRegression(max_iter=1000),
                                    {"clf__C": [0.01, 0.1, 1, 10, 100]}),
}
best_params = {}
for name, (clf, grid) in grids.items():
    gs = GridSearchCV(make_pipe(clf), grid, cv=skf, scoring="accuracy", n_jobs=-1)
    gs.fit(X_train, y_train)
    pipe = gs.best_estimator_
    best_params[name] = {k.replace("clf__", ""): v for k, v in
gs.best_params_.items()}
    yp = pipe.predict(X_test); s = pipe.predict_proba(X_test)[:, 1]
    fitted[name], preds[name], scores[name] = pipe, yp, s
    rows.append({
        "Model": name, "CV_Accuracy": gs.best_score_,
        "Accuracy": accuracy_score(y_test, yp), "Precision":
precision_score(y_test, yp),
        "Recall": recall_score(y_test, yp), "F1": f1_score(y_test, yp),
        "ROC_AUC": roc_auc_score(y_test, s),
    })
    r = rows[-1]
    print(f"{name:28s} best={best_params[name]} cv={r['CV Accuracy']:.4f}
acc={r['Accuracy']:.4f} "
          f"prec={r['Precision']:.4f} rec={r['Recall']:.4f} f1={r['F1']:.4f}")

res_df = pd.DataFrame(rows).sort_values(["CV_Accuracy", "F1"],
ascending=False).reset_index(drop=True)
print("\n", res_df.round(4).to_string(index=False), "\n")
res_df.to_csv("model_comparison.csv", index=False)

# ---------------------------------------------------------------- 6. Best model
and plots
best_name = res_df.loc[0, "Model"]
best = fitted[best_name]
print("Best model (highest cross-validated accuracy):", best_name, "\n")

plt.figure(figsize=(9, 4.8))
x = np.arange(len(res_df)); w = 0.2
for i, m in enumerate(["Accuracy", "Precision", "Recall", "F1"]):
    plt.bar(x + i * w, res_df[m], w, label=m)
plt.xticks(x + 1.5 * w, res_df["Model"], rotation=18, ha="right", fontsize=8)
plt.ylim(0.4, 1.02); plt.title("Model Comparison (test set)");
plt.legend(loc="lower right", fontsize=8)
plt.tight_layout(); plt.savefig("fig_model_comparison.png", dpi=150); plt.close()

cm = confusion_matrix(y_test, preds[best_name])
plt.figure(figsize=(5, 4))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
            xticklabels=["Rejected", "Approved"], yticklabels=["Rejected",
"Approved"])
plt.title(f"Confusion Matrix - {best_name}", fontsize=10)
plt.xlabel("Predicted"); plt.ylabel("Actual")
plt.tight_layout(); plt.savefig("fig_confusion_matrix.png", dpi=150); plt.close()

plt.figure(figsize=(6, 4.8))
for name in ["Logistic Regression", "SVM (RBF)", "Random Forest", "Gradient
Boosting", best_name]:
    fpr, tpr, _ = roc_curve(y_test, scores[name])
    plt.plot(fpr, tpr, label=f"{name} (AUC={roc_auc_score(y_test,
scores[name]):.3f})")
plt.plot([0, 1], [0, 1], "k--", alpha=0.4)
plt.xlabel("False Positive Rate"); plt.ylabel("True Positive Rate");
plt.title("ROC Curves")
plt.legend(fontsize=7); plt.tight_layout(); plt.savefig("fig_roc_curves.png",
dpi=150); plt.close()
```

```
report = classification_report(y_test, preds[best_name], target_names=["Rejected",
"Approved"], digits=4)
print(report)

# feature importance from a Random Forest fitted on the same pipeline
rf_pipe = fitted["Random Forest"]
feat names = rf pipe.named steps["pre"].get feature names out()
imp = pd.Series(rf_pipe.named_steps["clf"].feature_importances_, index=feat_names)
imp.index = [n.replace("num__", "").replace("cat__", "") for n in imp.index]
top = imp.sort_values(ascending=False).head(10)
print("Top Random Forest features:\n", top.round(4), "\n")
plt.figure(figsize=(7.5, 4.5))
top[::-1].plot.barh(color=BLUE)
plt.title("Top 10 Feature Importances (Random Forest)"); plt.xlabel("Importance")
plt.tight_layout(); plt.savefig("fig_feature_importance.png", dpi=150);
plt.close()

# stability of the best model on the full data
cv full = cross val score(best, X, y, cv=skf, scoring="accuracy")
print("5-fold CV accuracy on full data:", cv_full.round(4), "mean",
round(cv_full.mean(), 4), "\n")

# ------------------------------------------------------------------ 7. Save
joblib.dump(best, "loan_model.joblib")
tn, fp, fn, tp = cm.ravel()
summary = {
    "rows": int(len(df)), "approved": int(y.sum()), "rejected": int((1 -
y).sum()),
    "train": int(len(X_train)), "test": int(len(X_test)), "baseline":
round(float(baseline), 4),
    "best_model": best_name, "best_params": best_params,
    "confusion": {"tn": int(tn), "fp": int(fp), "fn": int(fn), "tp": int(tp)},
    "report": report, "results": res_df.round(4).to_dict(orient="records"),
    "cv_full": cv_full.round(4).tolist(), "cv_full_mean":
round(float(cv_full.mean()), 4),
    "top features": {k: round(float(v), 4) for k, v in top.items()},
    "missing": {k: int(v) for k, v in missing[missing > 0].items()},
    "approval_rates": rates,
}
with open("metrics.json", "w") as f:
    json.dump(summary, f, indent=2)



# ------------------------------------------------------------------ 8. Predict
for new applicants
def predict_applicant(applicant: dict):
    """applicant: dict with the raw columns of the dataset (without Loan_ID /
Loan_Status)."""
    a = pd.DataFrame([applicant])
    a["Dependents"] = a["Dependents"].replace("3+", "3").astype(float)
    a["TotalIncome"] = a["ApplicantIncome"] + a["CoapplicantIncome"]
    a["TotalIncome log"] = np.log1p(a["TotalIncome"])
    a["LoanAmount_log"] = np.log1p(a["LoanAmount"])
    a["EMI"] = a["LoanAmount"] / a["Loan_Amount_Term"]
    a["BalanceIncome"] = a["TotalIncome"] - a["EMI"] * 1000
    p = best.predict_proba(a)[0, 1]
    return ("APPROVED" if p >= 0.5 else "REJECTED"), p



samples = [
    ("Graduate, married, good credit, income 6000", dict(Gender="Male",
Married="Yes", Dependents="1", Education="Graduate",
        Self_Employed="No", ApplicantIncome=6000, CoapplicantIncome=1500,
LoanAmount=130,
        Loan_Amount_Term=360, Credit_History=1.0, Property_Area="Semiurban")),
    ("Same profile but bad credit history", dict(Gender="Male", Married="Yes",
Dependents="1", Education="Graduate",
        Self_Employed="No", ApplicantIncome=6000, CoapplicantIncome=1500,
LoanAmount=130,
```

```
        Loan_Amount_Term=360, Credit_History=0.0, Property_Area="Semiurban")),
    ("Low income, large loan, no credit record", dict(Gender="Female",
Married="No", Dependents="0", Education="Not Graduate",
        Self_Employed="Yes", ApplicantIncome=1800, CoapplicantIncome=0,
LoanAmount=250,
        Loan_Amount_Term=180, Credit_History=0.0, Property_Area="Rural")),
    ("High income, small loan, good credit", dict(Gender="Female", Married="Yes",
Dependents="0", Education="Graduate",
        Self_Employed="No", ApplicantIncome=9000, CoapplicantIncome=3000,
LoanAmount=120,
        Loan_Amount_Term=360, Credit_History=1.0, Property_Area="Urban")),
]
print("FINAL PREDICTIONS")
print("-" * 78)
for desc, app in samples:
    label, prob = predict_applicant(app)
    print(f"[{label:9s}] P(approve)={prob:.2f}  {desc}")
```